Grade 12 Information Technology — Study Guide

A plain-English study guide covering the **theory** and the **programming** (both **Java** and **Delphi**) that a Grade 12 IT learner works with. Use the contents list to jump to what you need. Each idea is explained simply, with a short example.

> **Tip:** You only code in ONE language for your exams (either Java *or* Delphi). Both are shown here so you can use whichever your school teaches. The *theory* is exactly the same for everyone.

Contents

PART A — THEORY

1. Hardware & System Technologies

Hardware = the physical parts of a computer. Grouped by what they do:

Category	Examples	Job
Input	keyboard, mouse, scanner, microphone, camera	Get data *into* the computer
Processing	CPU (processor), GPU	Do the actual work / calculations
Memory	RAM, cache	Hold data the CPU is busy with (temporary)
Storage	SSD, hard drive (HDD), flash drive	Keep data permanently
Output	monitor, printer, speakers	Show/give results *out*

Key points to know:

- **CPU** — the "brain". Speed measured in **GHz**. Has **cores** (more cores = can do more at once) and **cache** (very fast memory inside the CPU).

- **RAM** — temporary working memory. More RAM = run more programs smoothly. It is **volatile** (loses everything when power is off).
- **Storage** — permanent (**non-volatile**). **SSD** is much faster than a traditional **HDD**.
- **Virtual memory** — when RAM is full, the computer uses part of the storage as "pretend" RAM (slower).
- **Cache** — small, super-fast memory that stores frequently used data.
- **Moore's Law** — the idea that computing power roughly doubles every ~2 years.
- **GIGO** — "Garbage In, Garbage Out": bad input data gives bad output.

2. Software & System Concepts

Software = the programs that tell the hardware what to do.

- **System software** — runs the computer itself:
  - **Operating System (OS)** — e.g. Windows, macOS, Linux, Android. Its jobs: manage hardware, manage memory, manage files, manage processes, provide a user interface, manage security.
  - **Utility software** — small helper tools: antivirus, disk clean-up, backup, file compression.
  - **Drivers** — let the OS talk to specific hardware (e.g. a printer driver).
- **Application software** — what *you* use: browsers, word processors, games.

Other concepts:

- **Freeware** (free), **shareware** (free trial then pay), **proprietary** (paid, closed e.g. Windows), **open-source** (code is public, e.g. Linux).
- **Compiler vs Interpreter** — a *compiler* translates the whole program to machine code once (Java/Delphi mostly compile); an *interpreter* translates and runs line by line (e.g. Python).
- **Firmware** — software stored permanently on a hardware chip (e.g. BIOS).

3. Networks & Communication

A **network** is two or more devices connected to share data and resources.

Types by size:

Type	Meaning	Example
PAN	Personal Area Network	your phone + earbuds (Bluetooth)
LAN	Local Area Network	a school computer lab
WAN	Wide Area Network	the internet; branches in different cities

Common network devices:

- **Router** — connects networks together (e.g. your home to the internet).
- **Switch** — connects devices *within* one network.
- **NIC** (Network Interface Card) — lets a device join a network.
- **Access point** — provides Wi-Fi.
- **Modem** — connects to your internet provider.

Transmission media:

- **Wired** — UTP cable, fibre-optic (fastest, uses light).
- **Wireless** — Wi-Fi, Bluetooth, cellular (3G/4G/5G).

Other terms:

- **Bandwidth** — how much data can travel per second (measured in Mbps/Gbps).

- **Bottleneck** — the slowest part that limits overall speed.
- **Protocol** — a set of rules for communication (see the internet section).
- **Client-server** — powerful *servers* provide services to *client* devices.
- **Peer-to-peer** — devices share directly with each other (no central server).

4. The Internet & Web Technologies

- **Internet** — the huge global network connecting everything.
- **WWW (World Wide Web)** — the web pages/sites you view *on* the internet.
- **ISP** — Internet Service Provider (the company you pay for internet).
- **IP address** — a device's unique number on a network.
- **URL** — a web address (e.g. https://www.example.com).
- **DNS** — turns a URL into an IP address (like a phone book).

Protocols (rules) to know:

Protocol	Used for
HTTP / HTTPS	Loading web pages (HTTPS = secure)
FTP	Transferring files
SMTP / POP3 / IMAP	Sending / receiving email
TCP/IP	The core rules that run the internet

Modern concepts:

- **Cloud computing** — storing/running things on the internet instead of your own computer (e.g. Google Drive, Dropbox). Pros: access anywhere, backups. Cons: needs internet, privacy concerns.
- **IoT (Internet of Things)** — everyday devices connected to the internet (smart fridge, smart watch, smart doorbell).
- **Bandwidth vs shaping/throttling** — providers can slow (throttle) certain traffic.
- **HTML** — the language used to build web pages (tags like `<h1>`, `<p>`).

5. Data & Information Management (Databases)

- **Data** = raw facts (e.g. "14", "Thabo"). **Information** = data that has been processed to be useful (e.g. "Thabo is 14 years old").
- A **database** stores lots of related data in an organised way.

Database structure:

Term	Meaning
Table	A set of data about one thing (e.g. Students)
Record (row)	One item (one student)
Field (column)	One piece of info (e.g. Name)
Primary key	A field that *uniquely* identifies each record (e.g. StudentID)
Foreign key	A field that links to the primary key of another table

- **Relationships** — tables can be linked (one-to-many is most common, e.g. one class has many students).
- **Data integrity** — data is correct, complete and consistent.

- **Validation** (is the data *reasonable*? e.g. age 0–120) vs **Verification** (is the data *entered correctly*? e.g. typing a password twice).
- **Redundancy** — the same data stored many times (bad; wastes space and causes errors). Good database design reduces this (**normalisation**).
- **Big data / data warehousing** — storing and analysing huge amounts of data.

(SQL — the language to work with databases — is in Part C.)

6. Social & Ethical Implications

Security threats:

- **Malware** — bad software: **virus**, **worm**, **trojan**, **ransomware** (locks your files for money), **spyware**.
- **Phishing** — fake emails/sites that trick you into giving passwords.
- **Hacking** — gaining unauthorised access.
- **Social engineering** — manipulating *people* to break security.

Safeguards (protection):

- Antivirus software, **firewall**, strong **passwords**, **2FA** (two-factor authentication), **encryption**, regular **backups**, software updates, being careful with links/attachments.

Ethics & law:

- **Piracy** — illegally copying software/media.
- **Copyright** — the creator owns their work; you may not copy without permission.
- **Software licence** — the rules for how you may use a program.
- **Plagiarism** — presenting someone's work as your own.
- **POPIA** (South Africa) — the law that protects people's personal information; organisations must keep your data safe and use it fairly.

Health & environment:

- **Ergonomics** — setting up your desk/chair/screen to avoid injury (e.g. RSI, eye strain).
- **Green computing / e-waste** — recycle old electronics; save power.

7. Systems Development & Algorithms

- **SDLC (Software Development Life Cycle)** — the stages of making software:
    1. **Analysis** (understand the problem)
    2. **Design** (plan it)
    3. **Coding** (build it)
    4. **Testing** (check it works)
    5. **Implementation** (release it)
    6. **Maintenance** (fix/improve it)
- **Algorithm** — a step-by-step set of instructions to solve a problem.
- **Pseudocode** — writing an algorithm in plain, structured English (not real code). *This is what your PAT Technical document uses.*
- **Flowchart** — a diagram of an algorithm using shapes (start/stop, process, decision).

PART B — PROGRAMMING (Java + Delphi)

For each idea below: a short explanation, then the same thing in **Java** and in **Delphi**. Learn the one your school uses, but the *thinking* is identical.

8. Program structure

Java:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello world");
    }
}
```

Delphi (console):

```
program HelloWorld;
begin
  Writeln('Hello world');
end.
```

Note Delphi uses `begin ... end` where Java uses `{ ... }`.

9. Variables & data types

A **variable** is a named box that stores a value. A **data type** says what kind of value it holds.

Meaning	Java	Delphi
Whole number	`int`	`Integer`
Decimal number	`double`	`Real` / `Double`
Single character	`char`	`Char`
Text	`String`	`String`
True/false	`boolean`	`Boolean`

Java:

```
int age = 14;
double price = 99.50;
String name = "Aisha";
boolean isMember = true;
```

Delphi:

```
var
  age: Integer;
  price: Real;
  name: String;
  isMember: Boolean;
```

```
begin
  age := 14;
  price := 99.50;
  name := 'Aisha';
  isMember := True;
end;
```

Note: Delphi uses `:=` to assign, and single quotes for text.

10. Input & output

Java (console):

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = sc.nextLine();
System.out.println("Hello " + name);
```

Delphi (using GUI components):

```
// Reading from a text box called Edit1, showing a message:
var name: String;
begin
  name := Edit1.Text;
  ShowMessage('Hello ' + name);
end;
```

- Java: `nextInt()`, `nextDouble()`, `nextLine()`.
- Delphi GUI: read `Edit1.Text` (a String) and convert with `StrToInt`, `StrToFloat`; show with `ShowMessage` or set `Label1.Caption`.

11. Operators

Purpose	Java	Delphi
Add / subtract / multiply	`+ - *`	`+ - *`
Divide (decimal)	`/`	`/`
Whole-number divide	`/` (on ints)	`div`
Remainder (modulus)	`%`	`mod`
Equal to	`==`	`=`
Not equal	`!=`	`<>`
And / Or / Not	`&&		!	

12. Decisions (if / case)

Java:

```
if (age >= 18) {
    System.out.println("Adult");
} else if (age >= 13) {
    System.out.println("Teen");
} else {
    System.out.println("Child");
}

switch (belt) {
    case "White": System.out.println("Beginner"); break;
    default: System.out.println("Other"); break;
}
```

Delphi:

```
if age >= 18 then
  ShowMessage('Adult')
else if age >= 13 then
  ShowMessage('Teen')
else
  ShowMessage('Child');

case grade of
  1: ShowMessage('Beginner');
  2: ShowMessage('Intermediate');
  else ShowMessage('Other');
end;
```

13. Loops

Loops repeat code.

Java:

```
for (int i = 1; i <= 5; i++) {          // runs a set number of times
    System.out.println(i);
}

int n = 1;
while (n <= 5) {                         // runs while a condition is true
    System.out.println(n);
    n++;
}
```

Delphi:

```
var i: Integer;
begin
  for i := 1 to 5 do                     // set number of times
    Writeln(i);
```

```
  i := 1;
  while i <= 5 do                       // while a condition is true
  begin
    Writeln(i);
    i := i + 1;
  end;

  i := 1;
  repeat                                // runs at least once
    Writeln(i);
    i := i + 1;
  until i > 5;
end;
```

14. Methods (procedures & functions)

A reusable block of code. A **function** returns a value; a **procedure / void method** does not.

Java:

```
// returns a value
public int addNumbers(int a, int b) {
    return a + b;
}

// returns nothing (void)
public void greet(String name) {
    System.out.println("Hi " + name);
}
```

Delphi:

```
// function returns a value
function AddNumbers(a, b: Integer): Integer;
begin
  Result := a + b;
end;

// procedure returns nothing
procedure Greet(name: String);
begin
  ShowMessage('Hi ' + name);
end;
```

15. Strings (text handling)

Task	Java	Delphi
Length	`s.length()`	`Length(s)`

Task	Java	Delphi
Upper case	`s.toUpperCase()`	`UpperCase(s)`
Lower case	`s.toLowerCase()`	`LowerCase(s)`
Part of a string	`s.substring(0, 3)`	`Copy(s, 1, 3)`
Find text	`s.indexOf("x")`	`Pos('x', s)`
Remove spaces	`s.trim()`	`Trim(s)`
Number → text	`String.valueOf(n)`	`IntToStr(n)`
Text → number	`Integer.parseInt(s)`	`StrToInt(s)`

Note: Java counts characters from **0**, Delphi's `Copy` counts from **1**.

16. Arrays & lists

An **array** stores many values of the same type under one name.

Java:

```
String[] belts = {"White", "Yellow", "Green"};
System.out.println(belts[0]);            // White (starts at 0)

// A growable list (ArrayList) is used a lot in Java:
ArrayList<String> names = new ArrayList<>();
names.add("Sam");
```

Delphi:

```
var belts: array[0..2] of String;
begin
  belts[0] := 'White';
  belts[1] := 'Yellow';
  belts[2] := 'Green';
  ShowMessage(belts[0]);
end;
```

Loop through an array to process every item (e.g. add up marks, search for a name).

17. Object-Oriented Programming (OOP)

OOP is about building programs from **objects**. This is the heart of the PAT.

- **Class** — a blueprint (e.g. `Student`).
- **Object** — an actual thing made from the class (e.g. one specific student).
- **Attributes/fields** — the data (name, age).
- **Methods** — the actions the object can do.
- **Encapsulation** — keep fields **private** and use **get/set** methods to reach them (information hiding).
- **Constructor** — a special method that creates and sets up an object.
- **Inheritance** — a class can reuse another class's code (a `Child` class extends a `Parent` class).

- **Polymorphism** — the same method name can behave differently in different classes (e.g. many shapes each have their own `area()`).

Java:

```
public class Student {
    private String name;              // private field (encapsulation)

    public Student(String name) {     // constructor
        this.name = name;
    }
    public String getName() {         // accessor (get)
        return name;
    }
    public void setName(String name) {// mutator (set)
        this.name = name;
    }
}

// using it:
Student s = new Student("Aisha");
System.out.println(s.getName());
```

Delphi:

```
type
  TStudent = class
  private
    FName: String;                    // private field
  public
    constructor Create(AName: String);
    function GetName: String;         // accessor
    procedure SetName(AName: String); // mutator
  end;

constructor TStudent.Create(AName: String);
begin
  FName := AName;
end;

function TStudent.GetName: String;
begin
  Result := FName;
end;

procedure TStudent.SetName(AName: String);
begin
  FName := AName;
end;
```

18. Error handling

Stops the program crashing when something goes wrong (e.g. bad input, missing file). This is **defensive programming**.

Java:

```
try {
    int age = Integer.parseInt(input);   // may fail if not a number
} catch (NumberFormatException e) {
    System.out.println("Please enter a number.");
}
```

Delphi:

```
try
  age := StrToInt(Edit1.Text);            // may fail if not a number
except
  on E: Exception do
    ShowMessage('Please enter a number.');
end;
```

19. File handling (saving to secondary storage)

Programs save data to files so it isn't lost when they close.

Java (text file):

```
// write
BufferedWriter w = new BufferedWriter(new FileWriter("data.txt"));
w.write("Aisha,14");
w.close();

// read
BufferedReader r = new BufferedReader(new FileReader("data.txt"));
String line = r.readLine();
r.close();
```

Delphi (text file):

```
var f: TextFile;
begin
  AssignFile(f, 'data.txt');
  Rewrite(f);              // create/overwrite for writing
  Writeln(f, 'Aisha,14');
  CloseFile(f);

  AssignFile(f, 'data.txt');
  Reset(f);                // open for reading
  Readln(f, line);
  CloseFile(f);
end;
```

20. Databases from code (SQL)

Both languages can connect to a database and run **SQL** commands (see Part C for SQL). The idea is the same:

1. Connect to the database.
2. Send an SQL command (e.g. `SELECT * FROM Students`).
3. Loop through the results.

- **Java** uses **JDBC** (`Connection`, `Statement`, `ResultSet`).
- **Delphi** uses data components (e.g. `TADOConnection`, `TADOQuery`).

21. Algorithms (searching & sorting)

You should be able to explain these in pseudocode:

Linear search — check each item one by one until you find it.

```
FOR each item in the list:
    IF item = target THEN
        found it → stop
```

Bubble sort — repeatedly swap neighbours that are in the wrong order until the list is sorted.

```
REPEAT until no swaps:
    FOR each pair of neighbours:
        IF left > right THEN swap them
```

Also know: **binary search** (only works on a *sorted* list — repeatedly check the middle and throw away half) and **selection sort**.

22. Defensive programming

Writing code that expects mistakes and handles them safely:

- **Validate** input (right type, sensible range, not empty).
- Use **try/catch** (Java) / **try/except** (Delphi) for risky actions (file access, converting text to numbers, dividing).
- Give **clear error messages**.
- Use dropdowns/limited choices so users can't type invalid values.

PART C — EXTRAS

23. SQL quick reference

SQL is the language used to work with databases.

```
-- Get data
SELECT * FROM Students;                          -- all columns
SELECT Name, Age FROM Students;                  -- some columns
SELECT * FROM Students WHERE Age > 14;           -- with a condition
```

```
SELECT * FROM Students ORDER BY Name;            -- sorted
SELECT * FROM Students ORDER BY Age DESC;        -- sorted, biggest first

-- Add a new record
INSERT INTO Students (Name, Age) VALUES ('Sam', 13);

-- Change existing records
UPDATE Students SET Age = 15 WHERE Name = 'Sam';

-- Delete records
DELETE FROM Students WHERE Name = 'Sam';
```

Useful extras: `AND`, `OR`, `LIKE` (pattern match, e.g. `WHERE Name LIKE 'A%'`), `COUNT()`, `SUM()`, `AVG()`, `MAX()`, `MIN()`, and joining tables with `JOIN`.

24. Exam technique

- **Read the question twice.** Underline what it actually asks.
- Watch the **verbs**: *list/name* (short), *explain/describe* (more detail), *discuss* (give both sides).
- Use the **mark allocation** as a guide (3 marks ≈ 3 points).
- For coding questions, **write the structure first** (loop/if), then fill it in.
- For "trace the code" questions, make a small **table of variable values** and update it line by line.
- Don't leave blanks — attempt everything.

25. Glossary of key terms

Term	Simple meaning
Algorithm	Step-by-step instructions to solve a problem
Variable	A named box that stores a value
Data type	The kind of value a variable holds
Loop	Code that repeats
Method / procedure / function	A reusable block of code
Class	A blueprint for making objects
Object	A thing created from a class
Encapsulation	Hiding data using private fields + get/set
Inheritance	A class reusing another class's code
Polymorphism	Same method name, different behaviour
Array	A list of values under one name
Primary key	Field that uniquely identifies a database record
Validation	Checking data is reasonable
Verification	Checking data was entered correctly
Compiler	Translates whole program to machine code

Term	Simple meaning
RAM	Fast temporary memory (volatile)
Protocol	Rules for communication
Encryption	Scrambling data so only authorised people can read it
SDLC	The stages of building software

Good luck — study a little every day, practise writing code by hand, and make sure you can explain your PAT program in your own words.